Projet de traitement automatique du langage

Détection automatique des e-mails indésirables

Rapport de recherche et de mise en œuvre

Données, apprentissage supervisé, intégration Mailpit et évaluation

Projet :	Détecteur de SPAM
Langage :	Python
Modèle :	TF-IDF + Naive Bayes
Date :	16 février 2026

Rapport rédigé en LaTeX

Résumé

Ce rapport décrit un système local de détection d'e-mails indésirables. Le projet prend des messages étiquetés, nettoie leur contenu, construit une représentation TF-IDF et entraîne des classifieurs Naive Bayes. Le meilleur artefact est enregistré avec le vectoriseur, le classifieur et le seuil de décision. Il est ensuite chargé par une application qui observe les messages reçus dans Mailpit.

Le projet comporte aussi une évaluation reproductible. Les messages de test reçoivent un identifiant de groupe, appelé `run_id`, ou un identifiant individuel, appelé `message_id`. Le rapport peut donc analyser un seul message ou calculer une matrice de confusion sur un ensemble. Les tags affichés dans Mailpit sont volontairement courts, mais le journal conserve la probabilité de spam, le seuil, la confiance et les mots influents.

Les essais montrent que la chaîne logicielle fonctionne : un message normal, un spam évident et un groupe de quatre messages sont correctement traités. Un message normal au vocabulaire proche de celui d'un spam a cependant été classé spam avec une probabilité de 98,99 %. Cette erreur montre que la robustesse du système dépend autant des données que de l'algorithme.

Mots-clés : spam, ham, TF-IDF, Naive Bayes, précision, rappel, F1, Mailpit, matrice de confusion, audit.

Table des matières

Résumé **i**

1 Introduction **1**

1.1 Problématique 1

1.2 Question de recherche 1

1.3 Objectifs 1

1.4 Plan 2

2 Architecture du projet **3**

2.1 Chaîne globale 3

2.2 Arborescence 3

2.3 Responsabilité des composants 4

2.4 Interfaces 4

3 Données et corpus **5**

3.1 Sources d'apprentissage 5

3.2 Données de démonstration 5

3.3 Qualité des étiquettes 6

3.4 Données sensibles 6

4 Nettoyage et traitement linguistique **7**

4.1 Nettoyage 7

4.2 Tokenisation 7

4.3 Lemmatisation 7

4.4 Cohérence du pipeline 8
4.5 Export traité 8

5 TF-IDF et modèle probabiliste 9

5.1 Représentation TF-IDF 9
5.2 N-grammes 9
5.3 Naive Bayes 9
5.4 ComplementNB 10
5.5 Interprétation 10

6 Entraînement et choix du seuil 11

6.1 Séparation 11
6.2 Recherche d'hyperparamètres 11
6.3 Entraînement final 12
6.4 Pourquoi un seuil différent de 0,50 ? 12

7 Métriques et rapports 13

7.1 Matrice de confusion 13
7.2 Formules principales 13
7.3 Analyse individuelle 14
7.4 Analyse d'un run 14
7.5 Journal de secours 14

8 Intégration Mailpit 15

8.1 Envoi des messages 15
8.2 Surveillance 15
8.3 Etat et audit 15
8.4 Environnement de démonstration 16

9 Scénarios et résultats 17

9.1 Protocole 17
9.2 Cas normal 17

9.3 Cas spam . . . 17
9.4 Cas ambigu . . . 17
9.5 Run de quatre messages . . . 18
9.6 Synthèse . . . 18

10 Discussion, sécurité et limites **19**
10.1 Ce qui fonctionne . . . 19
10.2 Généralisation . . . 19
10.3 Confidentialité . . . 19
10.4 Décision humaine . . . 20
10.5 Prétraitement . . . 20

11 Conclusion **21**

A Lexique **22**

B Commandes et bibliographie **24**
B.1 Commandes . . . 24
B.2 Références . . . 24

Liste des tableaux

3.1 Schéma commun après harmonisation 5

6.1 Paramètres du modèle enregistré 12

Chapitre 1

Introduction

1.1 Problématique

Les e-mails indésirables représentent un problème de sécurité, de productivité et de confiance. Ils peuvent être publicitaires, frauduleux ou destinés à récupérer des informations confidentielles. Un filtre automatique doit distinguer un message légitime d'un message indésirable à partir de signaux textuels souvent imparfaits.

Le projet formule cette tâche comme une classification binaire. *ham*signifie qu'un message est normal ; *spam*signifie qu'il doit être considéré comme indésirable. Le système produit une classe et une probabilité estimée. Cette probabilité sert ensuite à appliquer un seuil et à mesurer la confiance.

1.2 Question de recherche

La question centrale est la suivante : peut-on construire une chaîne locale, reproductible et observable qui entraîne un modèle de spam, l'utilise sur de nouveaux messages et explique ses erreurs au moyen de statistiques ?

Cette question contient une dimension algorithmique et une dimension logicielle. Il faut sélectionner une représentation du langage et un classifieur, mais aussi organiser les fichiers, gérer les messages, conserver l'état et produire des rapports fiables.

1.3 Objectifs

Les objectifs sont :

— fusionner deux corpus d'e-mails ;

— nettoyer les en-têtes, les URLs, le HTML et les adresses ;
— transformer les textes en vecteurs TF-IDF ;
— comparer deux variantes de Naive Bayes ;
— sélectionner alpha et le seuil avec F_2 ;
— intégrer le modèle à Mailpit ;
— évaluer un message ou un groupe de messages ;
— conserver des journaux destinés au diagnostic.

1.4 Plan

Le rapport présente d'abord l'architecture, puis les données et le prétraitement. Il explique ensuite TF-IDF, Naive Bayes, la sélection du seuil et les métriques. Une partie est consacrée à Mailpit et aux scénarios réels exécutés. Les limites, les questions de sécurité et les améliorations scientifiques terminent l'étude.

Chapitre 2

Architecture du projet

2.1 Chaîne globale

La chaîne d'apprentissage s'exécute avec `Spam.py`. Elle produit le fichier `models/spam_tfidf_nb.joblib`. La chaîne de démonstration se trouve dans `Mailpit`. Elle comporte trois scripts : `app.py`, `send.py` et `evaluate.py`.

```
Datasets CSV
    |
    v
Spam.py -> nettoyage -> TF-IDF -> Naive Bayes
    |
    v
models/spam_tfidf_nb.joblib
    |
    v
SMTP Mailpit -> app.py -> tags et journaux
    |
    v
evaluate.py -> metriques
```

2.2 Arborescence

```
Détecteur de SPAM/
  README.md
  requirements.txt
  Spam.py
```

```
data/raw/
data/processed/
models/spam_tfidf_nb.joblib
Mailpit/app.py
Mailpit/send.py
Mailpit/evaluate.py
Mailpit/data/
Mailpit/runtime/
Mailpit/bin/mailpit
Mailpit/bin/mailpit.exe
```

Le dossier `data/raw` conserve les sources originales. Le dossier `data/processed` contient une exportation après préparation. Le dossier `models` contient le seul modèle de référence. Le dossier `Mailpit/runtime` contient des sorties et non du code source.

2.3 Responsabilité des composants

Composant	**Rôle**
`Spam.py`	Entraînement, réglage et sauvegarde.
`app.py`	Surveillance, prédiction, tags et audit.
`send.py`	Messages modèles, messages manuels et CSV.
`evaluate.py`	Rapports et métriques.
`models`	Artefact utilisé en production locale.
`runtime`	Etat, compteurs et historique.

2.4 Interfaces

Le protocole SMTP de Mailpit écoute normalement sur le port 1025. L'API et l'interface web utilisent le port 8025. Le navigateur montre les messages, mais la classification Python communique avec l'API HTTP. Cette distinction permet de tester l'application même sans reproduire toutes les actions manuelles de l'interface.

Chapitre 3

Données et corpus

3.1 Sources d'apprentissage

Le fichier `data/raw/spam_assassin.csv` contient 5 796 messages. Le fichier `data/raw/spam_ham_datase` contient 5 171 messages. Le premier utilise `target` comme étiquette, le second utilise `label_num`. Le script renomme ces colonnes en `y`.

Table 3.1 – Schéma commun après harmonisation

Champ	Type	Signification
`text`	chaîne	contenu de l'e-mail
`y`	entier	0 pour ham, 1 pour spam

La première source compte 3 900 messages normaux et 1 896 spams. La seconde compte 3 672 messages normaux et 1 499 spams. Le corpus total compte donc 10 967 lignes avant le filtrage des textes vides. La classe normale est majoritaire.

3.2 Données de démonstration

Le dossier `Mailpit/data` contient des données plus petites. Le fichier `demo_mails.csv` contient huit messages équilibrés, quatre normaux et quatre spams. Le fichier `spam_dataset.csv` contient 1 000 lignes, réparties à parts égales dans la version observée.

Ces petits fichiers ne servent pas à remplacer le corpus d'apprentissage. Ils servent à vérifier rapidement l'envoi, le regroupement, les tags et les statistiques.

3.3 Qualité des étiquettes

Une étiquette erronée est apprise comme une règle. Si un message normal est marqué spam, le classifieur reçoit un signal contradictoire. La qualité du jeu de données doit donc être contrôlée avant d'optimiser l'algorithme.

Il faut également rechercher les doublons et les variantes d'un même message. La présence de messages presque identiques dans l'entraînement et le test peut donner un score artificiellement élevé.

3.4 Données sensibles

Les e-mails peuvent contenir des noms, adresses et informations commerciales. Les datasets doivent être anonymisés lorsque nécessaire. Mailpit limite le risque d'envoi vers l'extérieur, mais les fichiers locaux et le journal d'audit restent sensibles.

Chapitre 4

Nettoyage et traitement linguistique

4.1 Nettoyage

La fonction de nettoyage transforme le texte en minuscules. Elle supprime certaines lignes d'en-tête, les balises HTML, les URLs et les adresses électroniques. Les caractères non alphabétiques sont retirés et les espaces sont normalisés.

Cette étape rend le vocabulaire plus stable. Elle retire cependant des signaux potentiellement utiles : domaine d'une URL, présence d'une pièce jointe, adresse de l'expéditeur ou informations de routage. Le choix est donc un compromis entre réduction du bruit et perte d'information.

4.2 Tokenisation

La tokenisation transforme la chaîne en liste de mots en séparant sur les espaces. Elle est simple et rapide. Elle fonctionne bien avec le nettoyage préalable, mais elle traite imparfaitement les contractions, les langues multiples et certaines formes particulières.

4.3 Lemmatisation

Le script utilise NLTK et WordNet pour ramener certaines formes vers une base lexicale. Les ressources `punkt`, `wordnet` et `omw-1.4` sont téléchargées si nécessaire.

La lemmatisation réduit parfois la fragmentation du vocabulaire. Elle ne donne pas une compréhension sémantique du message : elle modifie seulement la forme des tokens.

4.4 Cohérence du pipeline

Une limite importante est présente dans la version étudiée. `Spam.py` lemmatise pendant l'entraînement, tandis que `app.py` ne reproduit pas exactement cette lemmatisation à l'inférence. Le modèle reste opérationnel, mais l'entrée d'entraînement et l'entrée de production ne sont pas parfaitement homogènes.

La correction recommandée consiste à partager une même fonction de prétraitement ou à sauvegarder un pipeline complet. Cette amélioration doit être réalisée avant une comparaison définitive de modèles.

4.5 Export traité

Le fichier `data/processed/spam_processed.csv` contient 10 460 lignes après le nettoyage et conserve les colonnes `text`, `y`, `clean`, `tokens`, `lemmatized` et `norm_text`. Cet export sert à l'audit et à la compréhension des transformations.

Chapitre 5

TF-IDF et modèle probabiliste

5.1 Représentation TF-IDF

Un classifieur de texte a besoin de nombres. TF-IDF attribue un poids aux termes selon leur présence dans un document et leur rareté dans le corpus :

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \log\left(\frac{1 + N}{1 + \mathrm{df}(t)}\right) + 1.$$

Ici, N est le nombre de documents et $\mathrm{df}(t)$ le nombre de documents contenant le terme. Un terme fréquent partout est peu discriminant ; un terme présent surtout dans une classe peut être utile.

5.2 N-grammes

Le vectoriseur utilise `ngram_range=(1,2)`. Il prend donc les mots isolés et les paires de mots. Les bigrammes permettent de représenter des expressions comme `claim now`, qui peuvent être plus informatives que `claim` ou `now` séparément.

Les paramètres `min_df=2` et `max_df=0.95` filtrent les termes trop rares ou trop fréquents. `sublinear_tf=True` limite l'effet des répétitions.

Le modèle actuel possède environ 153 507 caractéristiques. La matrice est creuse : chaque message utilise seulement une petite partie du vocabulaire.

5.3 Naive Bayes

Naive Bayes estime une classe à partir des probabilités des caractéristiques :

$$P(c \mid x_1, \ldots, x_n) \propto P(c) \prod_i P(x_i \mid c).$$

L'hypothèse d'indépendance est simplificatrice, mais elle est souvent efficace en classification de textes. Le paramètre alpha effectue un lissage et évite les probabilités nulles.

5.4 ComplementNB

Le projet compare `MultinomialNB` et `ComplementNB`. ComplementNB peut être intéressant lorsque les classes sont déséquilibrées. Les valeurs d'alpha testées sont :

$$\alpha \in \{0.1, 0.3, 0.5, 1.0, 2.0\}.$$

Le modèle choisi dans l'artefact observé est ComplementNB avec alpha égal à 0,1.

5.5 Interprétation

Les log-probabilités du classifieur permettent d'estimer les termes qui ont contribué à la décision. Ces mots influents sont utiles pour le diagnostic, mais ils ne constituent pas une explication causale de l'intention humaine.

Chapitre 6

Entraînement et choix du seuil

6.1 Séparation

Une graine aléatoire égale à 42 est utilisée. Une première séparation réserve 25 % des données au test. Une seconde réserve 20 % du reste à la validation. Les proportions approximatives sont 60 % entraînement, 15 % validation et 25 % test.

La séparation est stratifiée afin de conserver la proportion de ham et de spam. Le vectoriseur TF-IDF est ajusté uniquement sur l'entraînement. La validation et le test sont transformés avec le vocabulaire déjà appris.

6.2 Recherche d'hyperparamètres

Pour chaque classifieur et chaque alpha, le script calcule les probabilités sur la validation. Il teste des seuils compris entre 0,20 et 0,80 par pas de 0,05. Le meilleur couple est choisi avec F_2.

$$F_\beta = (1 + \beta^2)\frac{PR}{\beta^2 P + R}, \qquad \beta = 2.$$

Le choix de $\beta = 2$ donne deux fois plus d'importance au rappel qu'à la précision dans l'équilibre recherché. L'objectif est de manquer le moins possible de spams, tout en observant les faux positifs.

6.3 Entraînement final

Après la sélection, le modèle est réentraîné sur entraînement plus validation. Il est ensuite évalué sur le test. Le fichier joblib conserve le vectoriseur, le classifieur, le seuil, le nom du modèle, alpha et la métrique.

La configuration observée est :

TABLE 6.1 – Paramètres du modèle enregistré

Classifieur	ComplementNB
Alpha	0,1
Seuil spam	0,35
Métrique	F_2 de validation
N-grammes	1 à 2

6.4 Pourquoi un seuil différent de 0,50 ?

Une probabilité n'est pas obligée d'être convertie en décision à 0,50. Un seuil plus bas rend la détection plus sensible, mais augmente potentiellement les faux positifs. Un seuil plus haut réduit certaines alertes, mais peut laisser passer des spams. Le choix doit correspondre au coût métier des erreurs.

Chapitre 7

Métriques et rapports

7.1 Matrice de confusion

La convention du projet est :

	prédit normal	prédit spam
réel normal	TN	FP
réel spam	FN	TP

TN et TP sont les bonnes décisions. FP est un faux positif : un message normal bloqué. FN est un faux négatif : un spam manqué.

7.2 Formules principales

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Précision} = \frac{TP}{TP + FP} \qquad \text{Rappel} = \frac{TP}{TP + FN}$$

$$F_1 = 2\frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

L'accuracy décrit la proportion globale de décisions correctes. La précision décrit la fiabilité des alertes spam. Le rappel décrit la capacité à retrouver les spams. Le F1 équilibre précision et rappel.

7.3 Analyse individuelle

Pour un message :

```
python evaluate.py --message-id MESSAGE_ID
```

Le rapport affiche l'attendu, le prédit, la vérification, la probabilité spam, le seuil, la confiance et les mots influents.

7.4 Analyse d'un run

Pour un ensemble :

```
python evaluate.py --run-id RUN_ID --details
```

Le rapport affiche la matrice, l'accuracy, les scores de chaque classe et le support. L'option `-details` ajoute une ligne par message. Sans cette option, le rapport reste synthétique.

7.5 Journal de secours

L'évaluation lit les tags Mailpit et utilise aussi `runtime/predictions_log.jsonl`. Ce secours évite de marquer un message comme ignoré lorsque l'API renvoie le message mais pas toutes les informations de tag.

Chapitre 8

Intégration Mailpit

8.1 Envoi des messages

`send.py` possède trois modes. `templates` envoie des exemples normaux et spam. `message` envoie un message personnalisé. `csv` rejoue les lignes d'un fichier avec un intervalle.

Chaque message peut recevoir `X-Run-Id`. Lorsque l'étiquette attendue est connue, `X-Expected-Label` est ajoutée. Ces métadonnées sont utilisées pour l'évaluation mais ne servent pas à modifier la décision du modèle.

8.2 Surveillance

`app.py` interroge l'API Mailpit. Il récupère le contenu texte ou convertit le HTML. Il appelle le modèle, calcule la probabilité et ajoute les tags :

- `resultat:normal` ou `resultat:spam` ;
- `confiance:faible`, `confiance:moyenne` ou `confiance:forte` ;
- `verification:correcte` ou `verification:incorrecte`.

Les tags ont été simplifiés pour rendre l'interface compréhensible. Les informations techniques restent dans le journal.

8.3 Etat et audit

`state.json` contient les identifiants déjà traités. Il évite de retraiter les mêmes messages à chaque boucle. `metrics.json` contient les compteurs cumulés. `predictions_log.jsonl` contient une ligne par décision avec le message, le run, l'étiquette attendue, la classe, la probabilité, le seuil, la confiance et les termes influents.

8.4 Environnement de démonstration

Mailpit sépare la démonstration du vrai courrier. Le serveur SMTP local reçoit les messages sans les délivrer à un destinataire extérieur. Cette isolation est utile pour l'enseignement, les scénarios contrôlés et le diagnostic des erreurs.

Chapitre 9

Scénarios et résultats

9.1 Protocole

Quatre scénarios ont été exécutés : un ham évident, un spam évident, un ham ambigu et un groupe de quatre messages. Chaque message possède une étiquette attendue indépendante de la prédiction.

9.2 Cas normal

Le message `Team meeting update` était attendu comme ham. Le modèle a produit ham avec une probabilité spam de 0,61 %. La confiance était forte. La matrice du run est $[[1, 0], [0, 0]]$ et le résultat est correct.

Cette accuracy de 100 % ne mesure cependant pas le comportement sur la classe spam, car le run ne contient qu'un seul ham.

9.3 Cas spam

Le message annonçant un gain gratuit était attendu comme spam. Le modèle a produit spam avec une probabilité d'environ 99,80 %. Les termes influents étaient `won free`, `time offer` et `here now`. La matrice est $[[0, 0], [0, 1]]$.

9.4 Cas ambigu

Le message parlant de compte, de confirmation et de site web était attendu comme ham. Le modèle a produit spam avec une probabilité de 98,99 %. Les termes influents étaient

`website to`, `our service` et `confirm your`. La matrice est $[[0, 1], [0, 0]]$: c'est un faux positif.

Cet exemple est important. Le modèle est très sûr de sa décision, donc un petit changement du seuil ne suffira probablement pas. Il faut enrichir les exemples ham similaires ou ajouter des caractéristiques de contexte.

9.5 Run de quatre messages

Le run groupé contenait deux ham et deux spam. La matrice obtenue est :

$$\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}.$$

Les quatre messages sont évalués et aucun n'est ignoré. Le regroupement par `run_id` fonctionne donc correctement.

9.6 Synthèse

Sur les sept messages de ces scénarios, six sont corrects et un est un faux positif. La matrice combinée est :

$$\begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}.$$

L'accuracy vaut 85,7 %. La précision spam vaut 75 %, le rappel spam 100 % et le F1 spam 85,7 %. Le système détecte donc bien les exemples spam testés, mais il déclenche parfois à tort sur des messages normaux.

Chapitre 10

Discussion, sécurité et limites

10.1 Ce qui fonctionne

La chaîne complète fonctionne techniquement : envoi SMTP, réception Mailpit, lecture API, chargement du modèle, prédiction, tags, journalisation et rapport. L'arborescence sépare correctement les sources, le modèle, les scripts et les sorties.

Le rapport individuel est maintenant technique : il affiche la probabilité, le seuil, la confiance et les mots influents. Le rapport de run sépare les statistiques globales des détails par message.

10.2 Généralisation

Les scénarios sont utiles pour l'intégration mais trop petits pour représenter tous les e-mails. Il faut conserver un test indépendant plus large et éviter de modifier le modèle en regardant directement ses résultats de test.

La distribution déséquilibrée du corpus rend l'accuracy insuffisante. Le rappel spam et les faux positifs doivent être suivis en priorité.

10.3 Confidentialité

Les journaux peuvent contenir des identifiants et des sujets. Dans un contexte réel, il faudrait anonymiser les données, limiter la rétention et contrôler l'accès au dossier. Mailpit ne doit pas être exposé publiquement sans authentification et sans protection.

10.4 Décision humaine

Un filtre ne devrait pas supprimer automatiquement un message à partir d'une seule probabilité. Une zone de revue humaine est préférable. Le projet actuel fournit une confiance et une possibilité d'audit, mais une politique de production devrait préciser les actions associées à chaque niveau.

10.5 Prétraitement

La différence entre la lemmatisation à l'entraînement et le nettoyage à l'inférence doit être corrigée. Cette incohérence peut expliquer une partie des erreurs sur des formes morphologiques non vues de la même façon.

Chapitre 11

Conclusion

Le projet constitue une base fonctionnelle et pédagogique de classification de spam. Il montre comment passer d'un corpus à un modèle sauvegardé, puis d'un modèle à une démonstration observable dans Mailpit. Il montre aussi qu'un bon résultat sur des cas évidents ne garantit pas une bonne compréhension du contexte.

La conclusion expérimentale est nuancée : l'intégration est opérationnelle, le regroupement par run fonctionne et les statistiques sont disponibles, mais un faux positif fortement confiant subsiste. Le travail futur doit améliorer les données et la cohérence du prétraitement avant une utilisation de production.

Chapitre A

Lexique

Terme	Définition
Spam	E-mail indésirable ou frauduleux.
Ham	E-mail normal et légitime.
Classification binaire	Décision entre deux classes.
Apprentissage supervisé	Apprentissage sur des exemples étiquetés.
Dataset	Ensemble de données.
Label	Classe attendue d'un exemple.
Feature	Variable fournie au modèle.
Token	Unité produite par la tokenisation.
Lemmatisation	Réduction vers une forme lexicale.
N-gramme	Suite contiguë de n tokens.
TF-IDF	Poids combinant fréquence locale et rareté globale.
Matrice creuse	Matrice contenant surtout des zéros.
Naive Bayes	Classifieur probabiliste.
ComplementNB	Variante de Naive Bayes pour les textes.
Alpha	Paramètre de lissage.
Seuil	Probabilité utilisée pour décider spam.
Hyperparamètre	Paramètre réglé avant l'utilisation finale.
Entraînement	Apprentissage sur un sous-ensemble.
Validation	Réglage sur un sous-ensemble séparé.
Test	Evaluation finale sur des données gardées à part.
Fuite de données	Utilisation involontaire d'informations du test.
Stratification	Conservation des proportions de classes.
TP	Spam correctement détecté.
TN	Ham correctement reconnu.

Terme	**Définition**
FP	Ham classé spam.
FN	Spam classé ham.
Précision	Fiabilité des alertes spam.
Rappel	Part des spams retrouvés.
F1	Equilibre entre précision et rappel.
Accuracy	Proportion totale de bonnes décisions.
Artefact	Fichier sauvegardé contenant le modèle.
SMTP	Protocole d'envoi d'e-mails.
API	Interface entre programmes.
Watcher	Programme qui surveille les messages.
Tag	Etiquette ajoutée à un message.
Run ID	Identifiant commun à plusieurs messages.
Message ID	Identifiant d'un message.
Audit trail	Historique détaillé des décisions.

Chapitre B

Commandes et bibliographie

B.1 Commandes

```
# installation et entrainement
python -m pip install -r requirements.txt
python Spam.py

# Mailpit
cd Mailpit
./bin/mailpit
python app.py --reset-state

# tests et rapports
python send.py templates --kind both --count 2 --interval 1
python evaluate.py --run-id RUN_ID --details
python evaluate.py --message-id MESSAGE_ID
```

B.2 Références

Bibliographie

[1] C. D. Manning, P. Raghavan et H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[2] G. Salton et C. Buckley, « Term-weighting approaches in automatic text retrieval », *Information Processing & Management*, vol. 24, 1988.

[3] A. McCallum et K. Nigam, « A comparison of event models for Naive Bayes text classification », AAAI Workshop on Learning for Text Categorization, 1998.

[4] J. D. M. Rennie, L. Shih, J. Teevan et D. R. Karger, « Tackling the poor assumptions of Naive Bayes text classifiers », Proceedings of ICML, 2003.

[5] Scikit-learn Developers, *Documentation scikit-learn*, https://scikit-learn.org/stable/.

[6] NLTK Project, *Natural Language Toolkit Documentation*, https://www.nltk.org/.

[7] Mailpit Project, *Documentation de Mailpit*, https://mailpit.axllent.org/.